

PROJECT REPORT

CHIP-8 Emulator

A C++ virtual machine with an SFML 3 play and debug frontend

Emulator core: opcode implementation, fetch-decode-execute cycle and program flow

Core contributors (per source comments)

Awais - initialisation, arithmetic, drawing, keypad

Mustafa - execution cycle, flow control, ROM loading, timers

September 2026

Table of Contents

1. Introduction	3
1.1 Background	3
1.2 Objectives	3
1.3 Scope of This Report	3
2. Project Overview	3
2.1 Technology Stack	3
2.2 Repository Structure	4
2.3 Division of Work	4
2.4 Building and Running	4
3. CHIP-8 Machine Model	4
3.1 The Chip8 Class	5
3.2 Memory Map	6
3.3 Keypad	6
4. Software Architecture	6
5. Initialisation and ROM Loading	7
5.1 The Font Set	7
5.2 init() - Resetting the Machine	7
5.3 load() - Reading a ROM	8
6. The Fetch-Decode-Execute Cycle	9
6.1 Safety Helpers	10
6.2 Fetch	11
6.3 Decode	11
6.4 Execute - the Dispatch Switch	11
7. Opcode Implementation	13
7.1 Complete Opcode Reference	13
7.2 How the Program Counter Moves	14
7.3 Flow Control: jumps, calls, returns and skips	15
7.4 Arithmetic and Logic	16
7.5 Drawing: CLS and DXYN	17
7.6 Keypad Instructions	18
7.7 Index Register and Memory Instructions	19
7.8 Timers and Random Numbers	21
8. Execution Walkthroughs	21
8.1 IBM Logo ROM: First 14 Instructions	22
8.2 Call and Return	22
8.3 Carry Flag	23
9. Timing and Frontend Integration	23
10. SFML Graphical Frontend (Brief)	24
11. Testing and ROM Compatibility	25
12. Design Notes, Quirks and Limitations	25
Strengths	26
Possible Future Improvements	26
13. Conclusion	26
References	27

1. Introduction

1.1 Background

CHIP-8 is an interpreted programming language and virtual machine that appeared in the mid-1970s, originally for hobbyist microcomputers such as the COSMAC VIP. Programs are distributed as small binary files (ROMs) made of 16-bit instructions called opcodes. Because the machine is tiny (4 KB of memory, sixteen 8-bit registers, a 64x32 monochrome display and a 16-key keypad) it is the classic first project for anyone learning how a CPU works and how emulators are built.

An emulator for CHIP-8 does not translate the ROM to native code. Instead it reproduces the machine in software: it keeps the memory, registers, stack and screen as ordinary variables and runs the same loop a real CPU would run, namely **fetch** an instruction, **decode** it and **execute** it, over and over.

1.2 Objectives

- Implement a complete, self-contained CHIP-8 CPU core in C++ with no dependency on any graphics library.
- Support the standard instruction set (34 of the 35 classic opcodes; the machine-code call `0NNN` is intentionally rejected).
- Load and run real ROMs, including Pong, Tetris, Breakout, Space Invaders and others.
- Provide a usable front end with SFML 3: a game view, a debugger view showing live CPU state, keyboard input, and a buzzer.
- Fail safely: invalid memory access or an unsupported opcode stops the virtual CPU with a message instead of crashing the host program.

1.3 Scope of This Report

It concentrates on the emulator core, that is, initialisation, ROM loading, the fetch-decode-execute cycle and the implementation of every opcode, with source code and explanations. The SFML graphical front end is described briefly in Section 10. The execution traces shown in Section 8 were produced by compiling the repository's core files unchanged and running them headlessly, so the register values are real program output.

2. Project Overview

2.1 Technology Stack

Table 1: Technologies used

Item	Detail
Language	C++
Graphics / audio / input	SFML 3 (Graphics, Window, System, Audio modules)
Build system	Direct <code>g++</code> command, or CMake 3.16+ (<code>CMakeLists.txt</code>)
Toolchain in README	MinGW / MSYS2 on Windows (the fixed layout and minimise button target Windows)
Tests	<code>tests/FrontendTests.cpp</code> (integration tests using synthetic programs)

Item	Detail
Content	33 ROM files under <code>roms/</code> with matching control-hint text files under <code>controls/</code>

2.2 Repository Structure

Table 2: Source files and their responsibilities

File	Responsibility
<code>src/Chip8.h</code>	Declaration of the <code>Chip8</code> class: all machine state and the opcode-group methods
<code>src/Chip8.cpp</code>	Font set, <code>init()</code> , arithmetic/logic opcodes, drawing opcodes, keypad opcodes
<code>src/Chip8Execution.cpp</code>	<code>emulate_cycle()</code> (fetch/decode/dispatch), <code>flow_control()</code> , <code>random_number()</code> , safety helpers
<code>src/Chip8Memory.cpp</code>	ROM loader <code>load()</code> and the memory/index-register opcodes
<code>src/Chip8Timers.cpp</code>	Delay/sound timer opcodes and the 60 Hz <code>update_timers()</code>
<code>src/main.cpp</code>	Command-line parsing and the main loop
<code>src/Frontend.h/.cpp</code>	<code>FrontendState</code> and CPU/timer scheduling (<code>updateEmulation</code>)
<code>src/Input.h/.cpp</code>	SFML events to <code>key[16]</code> , pause, step, reset, speed controls
<code>src/Display.h/.cpp</code> , <code>UiLayout.h</code>	Window, play view, debug view, pixel-font text
<code>src/Audio.h/.cpp</code>	Generated 480 Hz buzzer driven by the sound timer
<code>src/Controls.h/.cpp</code>	Reads per-game control hints from <code>controls/*.txt</code>

2.3 Division of Work

Comments at the top of the core files record who wrote what. **Awais** implemented initialisation, the arithmetic/logic group, drawing and keypad handling (`Chip8.cpp`). **Mustafa** implemented the fetch-decode-execute cycle, flow control, ROM loading/memory instructions and timers (`Chip8Execution.cpp`, `Chip8Memory.cpp`, `Chip8Timers.cpp`). The SFML frontend was then layered on top without changing the core's public interface.

2.4 Building and Running

The README documents a direct compiler command (shown below) and an optional CMake build. ROMs start paused; press Space or click the Play button to begin.

Listing 1: Build and run (from README, PowerShell)

```
g++ -std=c++17 -Wall -Wextra -Wpedantic src/*.cpp -o build/chip8.exe \
    -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio
./build/chip8.exe "roms/Pong.ch8" # play
./build/chip8.exe "roms/Pong.ch8" --debug # open the debugger view
./build/chip8.exe "roms/Pong.ch8" --hz 900 # CPU speed (100-2000)
```

3. CHIP-8 Machine Model

The whole virtual machine lives in one class. Every hardware component of CHIP-8 maps to a data member, which is what makes the core easy to test and easy for a frontend to inspect.

Table 3: CHIP-8 hardware and its representation in the code

Component	CHIP-8 specification	Member in `Chip8`
Memory	4096 bytes; programs load at 0x200	<code>uint8_t memory[4096]</code>
Registers	V0-VF, 8-bit; VF doubles as flag	<code>uint8_t V[16]</code>
Index register	12-bit address register I	<code>uint16_t I</code>
Program counter	Address of next instruction	<code>uint16_t pc</code> (starts at 0x200)
Call stack	16 return addresses	<code>uint16_t stack[16]</code> , <code>uint8_t sp</code>
Display	64x32 monochrome pixels	<code>uint8_t gfx[64*32]</code> (row-major, 1 = lit)
Keypad	16 hex keys	<code>uint8_t key[16]</code> (1 = held)
Timers	Delay and sound, count down at 60 Hz	<code>delay_timer</code> , <code>sound_timer</code>
Current instruction	(debug aid)	<code>uint16_t opcode</code>
Control flags	(emulator only)	<code>drawFlag</code> , <code>running</code>

3.1 The Chip8 Class

Listing 2: Chip8.h - complete class declaration

```
class Chip8 {
public:
    uint8_t V[16] = {0};
    uint8_t memory[4096] = {0};
    uint8_t gfx[64 * 32] = {0};
    uint8_t key[16] = {0};
    uint16_t stack[16] = {0};

    uint16_t opcode = 0;
    uint16_t I = 0;
    uint16_t pc = 0x200;
    uint8_t sp = 0;
    uint8_t delay_timer = 0;
    uint8_t sound_timer = 0;
    bool drawFlag = false;
    bool running = true;

    void init();
    bool load(const char* file_path);
    void emulate_cycle();
    void update_timers();

    void arithmetic();
    void drawing();
    void keypad();
    void flow_control();
    void random_number();
    void memory_instructions();
    void timer_instructions();

private:
    void stop(const char* message);
    bool can_read_memory(uint16_t address, uint16_t length) const;
};
```

Notes on the design:

- All state is `public` so a frontend can read `gfx`, `V`, `pc` and the timers directly, and write `key[]`. The SFML code never re-implements an instruction.
- Opcodes are grouped by function (`arithmetic`, `drawing`, `keypad`, `flow_control`, `random_number`, `memory_instructions`, `timer_instructions`). `emulate_cycle()` decides which group handles the current opcode.

- `stop()` and `can_read_memory()` are **private** helpers used by every group for safe error handling.

3.2 Memory Map

Table 4: Memory layout used by the emulator

Address range	Size	Contents
0x000 - 0x04F	80 bytes	Unused (interpreter area on original hardware)
0x050 - 0x09F	80 bytes	Built-in hexadecimal font: 16 glyphs x 5 bytes
0x0A0 - 0x1FF	352 bytes	Unused
0x200 - 0xFFFF	3584 bytes	ROM program and its data (maximum ROM size = 3584 bytes)

3.3 Keypad

The CHIP-8 keypad is a 4x4 grid of hexadecimal keys. The frontend maps it to the left block of a QWERTY keyboard using **physical key positions** (scancodes), so the layout is correct on non-US keyboards too.

Table 5: Keypad mapping

Row	Physical keyboard	CHIP-8 key
1	1 2 3 4	1 2 3 C
2	Q W E R	4 5 6 D
3	A S D F	7 8 9 E
4	Z X C V	A 0 B F

4. Software Architecture

The project is split into a **core** (pure C++, no SFML) and a **frontend** (SFML). The core exposes its state and three entry points, `load()`, `emulate_cycle()` and `update_timers()`. The frontend decides *when* to call them and shows the result. This separation is the most important design decision in the project: the CPU logic can be compiled and tested with nothing but a C++ compiler, which is exactly how the traces in Section 8 were produced.

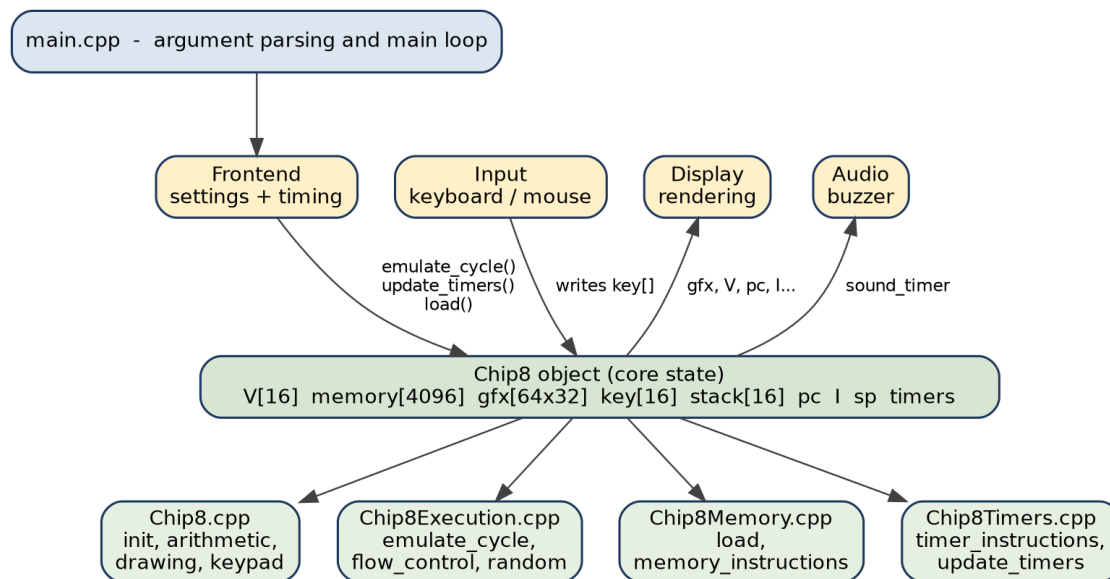


Figure 1: Module architecture: frontend drives and observes the core

Data flows in three directions. **Input** writes key states into `chip8.key[]`. **Frontend** (`updateEmulation`) calls `emulate_cycle()` and `update_timers()` at the right rates. **Display** and **Audio** only read state (`gfx`, registers, `sound_timer`), so the frontend cannot corrupt the emulation.

5. Initialisation and ROM Loading

5.1 The Font Set

CHIP-8 programs draw digits and letters by pointing the index register at a built-in glyph. Each glyph is a 4x5 pixel sprite stored as five bytes, one byte per row, using only the upper four bits. The emulator holds sixteen glyphs (0-F) in `chip8_fontset` and copies them into memory at 0x050 during initialisation.

Listing 3: Chip8.cpp - start of the font table

```

uint8_t chip8_fontset[80] =
{
    0xF0, 0x90, 0x90, 0x90, 0xF0, //0
    0x20, 0x60, 0x20, 0x20, 0x70, //1
    0xF0, 0x10, 0xF0, 0x80, 0xF0, //2
    ... // 16 glyphs = 80 bytes in total
}

```

Reading the glyph for **0**: `F0 90 90 90 F0` gives the rows `1111`, `1001`, `1001`, `1001`, `1111`, which draws a hollow rectangle shaped like a zero.

5.2 init() - Resetting the Machine

Listing 4: Chip8.cpp - Chip8::init()

```

void Chip8::init()
{
    pc = 0x200; // reset program counter
    opcode = 0; // reset opcode
    I = 0;      // reset I
    sp = 0;     // reset stack pointer
    running = true;
    drawFlag = true;

    // clear display
    for(int i = 0; i < (64 * 32); i++){

```

```

    gfx[i] = 0;
}

// clear stack, key and registers
for(int i = 0; i < 16; i++){
    stack[i] = 0;
    key[i] = 0;
    V[i] = 0;
}

// clear memory
for(int i = 0; i < 4096; i++){
    memory[i] = 0;
}

// load fontset into memory
for(int i = 0; i < 80; i++){
    memory[i + 0x050] = chip8_fontset[i];
}

// reset timers
delay_timer = 0;
sound_timer = 0;

// seed random generation
srand(time(NULL));
}

```

`init()` returns the virtual machine to power-on state:

- `pc = 0x200`: CHIP-8 programs always start executing at address 0x200.
- `running = true` and `drawFlag = true` so the first frame is drawn and the CPU accepts cycles.
- The screen, stack, keys, registers and the whole 4 KB memory are zeroed, then the font is copied to `memory[0x050 + i]`.
- Timers are cleared and `srand(time(NULL))` seeds the pseudo-random generator used by `CXNN`.

5.3 load() - Reading a ROM

Listing 5: `Chip8Memory.cpp` - `Chip8::load()`

```

bool Chip8::load(const char* file_path) {
    init();

    if (file_path == nullptr) {
        std::fprintf(stderr, "CHIP-8 error: ROM path is null\n");
        running = false;
        return false;
    }

    std::ifstream rom(file_path, std::ios::binary);
    if (!rom) {
        std::fprintf(stderr, "CHIP-8 error: could not open ROM '%s'\n", file_path);
        running = false;
        return false;
    }

    const std::vector<char> bytes((std::istreambuf_iterator<char>(rom)),
                                  std::istreambuf_iterator<char>());
    const std::size_t available = sizeof(memory) - 0x200;

    if (bytes.empty()) {
        stop("ROM file is empty");
        return false;
    }
    if (bytes.size() > available) {
        std::fprintf(stderr, "CHIP-8 error: ROM is too large (%zu bytes; maximum is %zu)\n",
                    bytes.size(), available);
        running = false;
        return false;
    }
}

```

```

    }
    for (std::size_t index = 0; index < bytes.size(); ++index) {
        memory[0x200 + index] = static_cast<uint8_t>(
            static_cast<unsigned char>(bytes[index]));
    }
    return true;
}

```

`load()` always calls `init()` first, so loading a ROM is also a full reset (the frontend's F5 key relies on this). It then validates the input before touching memory: a null path, an unreadable file, an empty file, or a file larger than 3584 bytes ($4096 - 0x200$) all set `running = false` and return `false`. Bytes are copied into `memory[0x200 + index]`. The double cast through `unsigned char` avoids sign-extension problems, because `char` may be signed.

6. The Fetch-Decode-Execute Cycle

`emulate_cycle()` executes exactly one instruction. It is called hundreds of times per second by the frontend (700 by default).

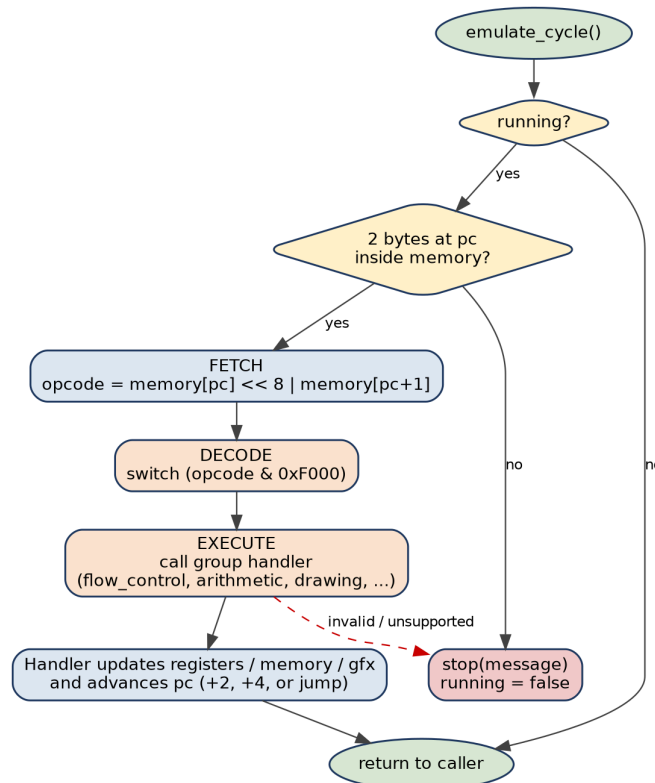


Figure 2: Flow of one `emulate_cycle()` call

6.1 Safety Helpers

Listing 6: `Chip8Execution.cpp` - `stop()` and `can_read_memory()`

```

void Chip8::stop(const char* message) {
    std::fprintf(stderr, "CHIP-8 error: %s\n", message);
    running = false;
}

bool Chip8::can_read_memory(uint16_t address, uint16_t length) const {
    return address < 4096 && length <= 4096 - address; //avoiding overflow
}

```

`stop()` prints a message to `stderr` and clears `running`; every later `emulate_cycle()` call returns immediately. `can_read_memory(address, length)` checks `address < 4096 && length <= 4096 - address`. The comment in the code notes that this form avoids integer overflow: writing `address + length <= 4096` on 16-bit values could wrap around and wrongly pass. Nearly every opcode that touches memory uses this helper.

6.2 Fetch

Listing 7: Chip8Execution.cpp - fetching the opcode

```
void Chip8::emulate_cycle() {
    if (!running) {
        return;
    }

    if (!can_read_memory(pc, 2)) {
        stop("program counter is outside memory");
        return;
    }

    opcode = static_cast<uint16_t>(memory[pc] << 8) | memory[pc + 1];
}
```

CHIP-8 opcodes are 2 bytes long and stored **big-endian** (high byte first). The code loads the byte at `pc`, shifts it left by 8 and ORs in the next byte. For example if `memory[0x200] = 0xA2` and `memory[0x201] = 0x2A`, then `opcode = 0xA22A`. The PC is validated first so a runaway jump cannot read past the end of memory.

6.3 Decode

The instruction is not decoded into separate variables up front. Instead the top nibble selects an instruction family and each handler extracts the fields it needs with bit masks:

Table 6: Opcode fields

Field	Extraction	Meaning
X	<code>(opcode & 0xF00) >> 8</code>	Index of register VX
Y	<code>(opcode & 0x0F0) >> 4</code>	Index of register VY
N	<code>opcode & 0x00F</code>	4-bit constant (sprite height, 8XY_sub-op)
NN	<code>opcode & 0x00FF</code>	8-bit constant
NNN	<code>opcode & 0x0FFF</code>	12-bit address

Worked example: `0x8AB4`. The top nibble `8` selects the arithmetic family, `X = A`, `Y = B`, and the last nibble `4` selects "add VY to VX with carry".

6.4 Execute - the Dispatch Switch

Listing 8: Chip8Execution.cpp - dispatch on the top nibble

```
switch (opcode & 0xF00) {
case 0x0000:
    if (opcode == 0x00E0 || opcode == 0x00EE) {
        if (opcode == 0x00E0) {
            drawing();
        } else {
            flow_control();
        }
    } else {
        stop("unsupported 0NNN instruction");
    }
    break;
case 0x1000:
```

```
case 0x2000:
case 0x3000:
case 0x4000:
case 0x5000:
case 0x9000:
case 0xB000:
    if ((opcode & 0xF000) == 0x5000 || (opcode & 0xF000) == 0x9000) {
        if ((opcode & 0x000F) != 0) {
            stop("unsupported 5XYN or 9XYN instruction");
            break;
        }
    }
    flow_control();
    break;
case 0x6000:
case 0x7000:
case 0x8000:
    arithmetic();
    break;
case 0xA000:
    memory_instructions();
    break;
case 0xC000:
    random_number();
    break;
case 0xD000:
    drawing();
    break;
case 0xE000:
    if ((opcode & 0x00FF) == 0x009E || (opcode & 0x00FF) == 0x00A1) {
        keypad();
    } else {
        stop("unsupported EX instruction");
    }
    break;
case 0xF000:
    switch (opcode & 0x00FF) {
        case 0x0007:
        case 0x0015:
        case 0x0018:
            timer_instructions();
            break;
        case 0x000A:
            keypad();
            break;
        case 0x001E:
        case 0x0029:
        case 0x0033:
        case 0x0055:
        case 0x0065:
            memory_instructions();
            break;
        default:
            stop("unsupported FX instruction");
            break;
    }
    break;
default:
    stop("unsupported instruction");
    break;
}
```

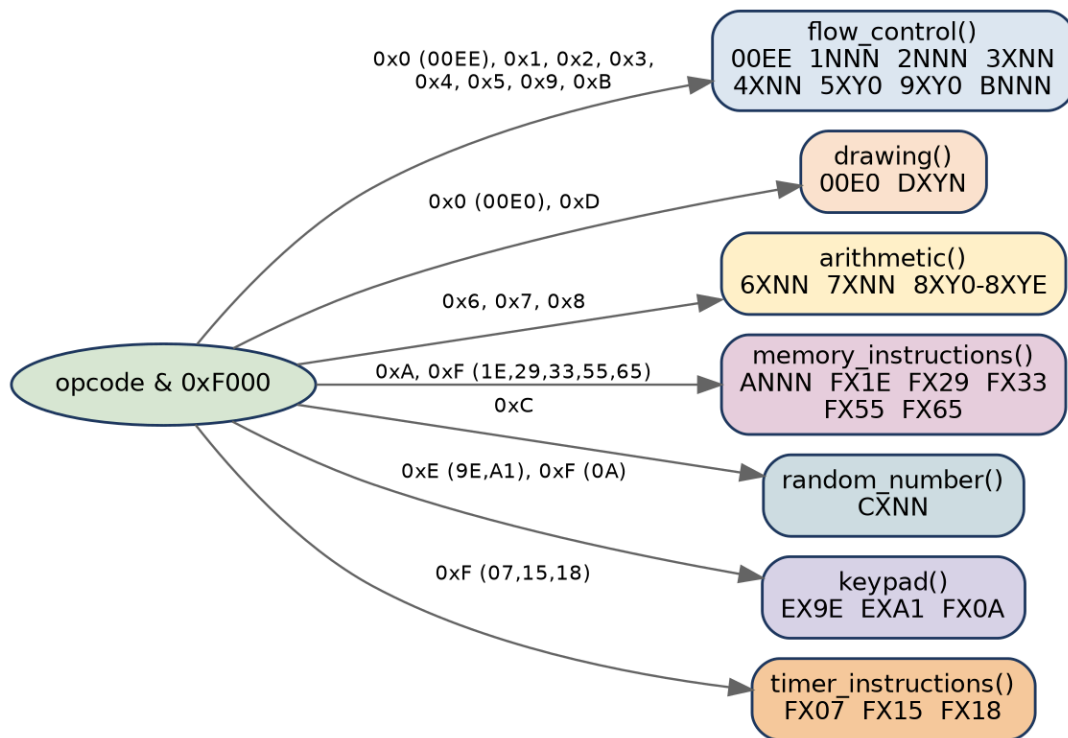


Figure 3: Which handler executes which opcode

Points worth noticing in the dispatcher:

- **Family 0x0000** is special. Only **00E0** (clear screen) and **00EE** (return) are accepted. Any other **0NNN** (a call to native machine code on the original hardware) is rejected with "unsupported 0NNN instruction".
- **5XY0 and 9XY0** must have a zero low nibble; otherwise the emulator halts. This catches corrupt or non-CHIP-8 code early.
- **Families 0xE and 0xF** are sub-decoded on the low byte (**opcode & 0x00FF**) to choose between keypad, timer and memory handlers.
- Anything unrecognised reaches a **default** branch and calls **stop()**. The emulator never silently ignores an unknown instruction.

7. Opcode Implementation

7.1 Complete Opcode Reference

The table lists all 34 implemented opcodes, the function that executes each one, and its effect on machine state. Notation: **VX/VY** are registers, **NN** an 8-bit value, **NNN** a 12-bit address.

Table 7: Implemented instruction set

Opcode	Mnemonic / meaning	Handler	Effect
00E0	CLS - clear screen	<code>drawing()</code>	Zero gfx; <code>drawFlag = true</code> ; <code>pc += 2</code>
00EE	RET - return from subroutine	<code>flow_control()</code>	<code>sp--</code> ; <code>pc = stack[sp] + 2</code>
1NNN	JP NNN - jump	<code>flow_control()</code>	<code>pc = NNN</code>
2NNN	CALL NNN - call subroutine	<code>flow_control()</code>	<code>stack[sp++] = pc</code> ; <code>pc = NNN</code>
3XNN	SE VX, NN - skip if equal	<code>flow_control()</code>	<code>pc += 4</code> if <code>VX == NN</code> , else 2

Opcode	Mnemonic / meaning	Handler	Effect
4XNN	SNE VX, NN - skip if not equal	flow_control()	pc += 4 if VX != NN, else 2
5XY0	SE VX, VY - skip if registers equal	flow_control()	pc += 4 if VX == VY, else 2
6XNN	LD VX, NN - load constant	arithmetic()	VX = NN
7XNN	ADD VX, NN - add constant	arithmetic()	VX += NN (wraps, VF unchanged)
8XY0	LD VX, VY	arithmetic()	VX = VY
8XY1	OR VX, VY	arithmetic()	VX = VY
8XY2	AND VX, VY	arithmetic()	VX &= VY
8XY3	XOR VX, VY	arithmetic()	VX ^= VY
8XY4	ADD VX, VY with carry	arithmetic()	VX = VX + VY; VF = 1 on carry
8XY5	SUB VX, VY	arithmetic()	VX = VX - VY; VF = (VX >= VY)
8XY6	SHR VX - shift right	arithmetic()	VX >>= 1; VF = shifted-out bit
8XY7	SUBN VX, VY (VY - VX)	arithmetic()	VX = VY - VX; VF = (VY >= VX)
8XYE	SHL VX - shift left	arithmetic()	VX <<= 1; VF = shifted-out bit
9XY0	SNE VX, VY - skip if not equal	flow_control()	pc += 4 if VX != VY, else 2
ANNN	LD I, NNN	memory_instructions()	I = NNN
BNNN	JP V0, NNN	flow_control()	pc = NNN + V0
CXNN	RND VX, NN	random_number()	VX = rand() & NN
DXYN	DRW VX, VY, N - draw sprite	drawing()	XOR N bytes at I onto screen; VF = collision
EX9E	SKP VX - skip if key pressed	keypad()	pc += 4 if key[VX], else 2
EXA1	SKNP VX - skip if key not pressed	keypad()	pc += 4 if !key[VX], else 2
FX07	LD VX, DT	timer_instructions()	VX = delay_timer
FX0A	LD VX, K - wait for key	keypad()	Stores first pressed key; otherwise pc unchanged (repeats)
FX15	LD DT, VX	timer_instructions()	delay_timer = VX
FX18	LD ST, VX	timer_instructions()	sound_timer = VX
FX1E	ADD I, VX	memory_instructions()	I += VX
FX29	LD F, VX - font glyph address	memory_instructions()	I = 0x050 + VX * 5
FX33	LD B, VX - BCD	memory_instructions()	Hundreds, tens, ones of VX to memory[I..I+2]
FX55	LD [I], VX - store registers	memory_instructions()	memory[I+i] = Vi for i = 0..X
FX65	LD VX, [I] - load registers	memory_instructions()	Vi = memory[I+i] for i = 0..X

7.2 How the Program Counter Moves

Every opcode is 2 bytes, so the default step is `pc += 2`. There are four patterns, and every handler follows one of them:

Table 8: Program-counter behaviour

Pattern	Used by	Result
Advance	Most instructions	pc += 2
Skip	3XNN, 4XNN, 5XY0, 9XY0, EX9E, EXA1	pc += 2 normally, pc += 4 when the condition holds (skips one whole instruction)
Jump / call	1NNN, 2NNN, BNNN	pc set to a new address (2NNN also pushes the old pc)

Pattern	Used by	Result
Return	00EE	<code>pc = stack[--sp] + 2</code> , the instruction after the original CALL
Hold	FX0A with no key pressed	<code>pc</code> unchanged, so the same instruction runs again next cycle

7.3 Flow Control: jumps, calls, returns and skips

Listing 9: Chip8Execution.cpp - flow_control() (call, return, skip)

```

void Chip8::flow_control() {
    const uint16_t nnn = opcode & 0x0FFF;
    const uint8_t x = static_cast<uint8_t>((opcode & 0x0F00) >> 8);
    const uint8_t y = static_cast<uint8_t>((opcode & 0x00F0) >> 4);
    const uint8_t nn = static_cast<uint8_t>(opcode & 0x00FF);

    switch (opcode & 0xF000) {
        case 0x0000:
            if (sp == 0) {
                stop("stack underflow on return");
                return;
            }
            --sp;
            if (stack[sp] > 4092) {
                stop("return address is outside memory");
                return;
            }
            pc = static_cast<uint16_t>(stack[sp] + 2);
            break;
        case 0x1000:
            if (!can_read_memory(nnn, 2)) {
                stop("jump address is outside memory");
                return;
            }
            pc = nnn;
            break;
        case 0x2000:
            if (sp >= 16) {
                stop("stack overflow on call");
                return;
            }
            if (!can_read_memory(nnn, 2)) {
                stop("call address is outside memory");
                return;
            }
            stack[sp] = pc;
            ++sp;
            pc = nnn;
            break;
        case 0x3000:
            pc = static_cast<uint16_t>(pc + (V[x] == nn ? 4 : 2));
            break;
        case 0x4000:
            pc = static_cast<uint16_t>(pc + (V[x] != nn ? 4 : 2));
            break;
        case 0x5000:
            pc = static_cast<uint16_t>(pc + (V[x] == V[y] ? 4 : 2));
            break;
        case 0x9000:
            pc = static_cast<uint16_t>(pc + (V[x] != V[y] ? 4 : 2));
            break;
        case 0xB000: {
            const uint16_t address = static_cast<uint16_t>(nnn + V[0]);
            if (!can_read_memory(address, 2)) {
                stop("BNNN target is outside memory");
                return;
            }
            pc = address;
            break;
        }
        default:
            stop("invalid flow-control instruction");
            break;
    }
}

```

CALL (2NNN) saves the address of the CALL instruction itself in `stack[sp]`, increments `sp` and jumps. Overflow beyond 16 entries stops the CPU ("stack overflow on call"). **RET (00EE)** does the reverse: it decrements `sp` (underflow is an error), reads the saved address and sets `pc = stack[sp] + 2`. Adding 2 moves past the CALL so the program continues with the next instruction instead of calling again forever.

Skip instructions use a compact idiom: `pc + (condition ? 4 : 2)`. A single expression handles both "condition false, advance normally" and "condition true, skip the next instruction". **BNNN** computes `NNN + V0` and, like the other jumps, validates the target is inside memory before jumping.

7.4 Arithmetic and Logic

Listing 10: `Chip8.cpp` - constants and the ALU group (excerpt)

```
void Chip8::arithmetic()
{
    // Implements 11 arithmetic opcodes
    int x = (opcode & 0x0F00) >> 8;
    int nn = opcode & 0x00FF;

    switch (opcode & 0xF000)
    {
        // 6XNN - Sets VX to NN
        case 0x6000:
            V[x] = nn;
            pc += 2;
            break;

        // 7XNN - Adds NN to VX
        case 0x7000:
            V[x] += nn;
            pc += 2;
            break;

        case 0x8000:
        {
            int y = (opcode & 0x00F0) >> 4;
            int vx = V[x];
            int vy = V[y];
            ... // 8XY0 - 8XY3 (assign, OR, AND, XOR)
            // 8XY4 - Add, with carry
            case 0x4:
                V[x] = vx + vy;
                V[0xF] = (vx + vy > 255);
                pc += 2;
                break;

            // 8XY5 - VX minus VY
            case 0x5:
                V[x] = vx - vy;
                V[0xF] = (vx >= vy);
                pc += 2;
                break;

            // 8XY6 - Shift VX right
            case 0x6:
                V[x] = vx >> 1;
                V[0xF] = vx & 1;
                pc += 2;
                break;

            // 8XY7 - VY minus VX
            case 0x7:
                V[x] = vy - vx;
                V[0xF] = (vy >= vx);
                pc += 2;
                break;

            // 8XYE - Shift VX left
            case 0xE:
                V[x] = vx << 1;
                V[0xF] = (vx >> 7) & 1;
                pc += 2;
                break;
        }
    }
}
```

6XNN and **7XNN** operate directly on $V[x]$. Because $V[x]$ is `uint8_t`, $V[x] += nn$ wraps modulo 256 automatically, matching real hardware. For the **8XY_** family the code first copies the operands into `int vx = V[x]; int vy = V[y];`. This is important for two reasons:

- **Correct carry/borrow detection.** $vx + vy$ is computed in `int`, so $vx + vy > 255$ reliably detects overflow before the sum is truncated to 8 bits.
- **Correct behaviour when X = F.** The flag is written to $V[0xF]$ *after* the result is written to $V[x]$. Using the saved copies means the flag calculation is not affected if VX and VF are the same register.

Note the flag semantics: for subtraction, $VF = 1$ means **no borrow** ($vx \geq vy$), which is the CHIP-8 convention, not the reverse. Shifts store the bit that falls off the register into VF.

Verified example. Running $V0 = 0xFF$; $V1 = 0x02$; **8014** gives $V0 = 0x01$ and $VF = 1$, because $255 + 2 = 257$ exceeds 8 bits (see Section 8.3).

7.5 Drawing: CLS and DXYN

Listing 11: Chip8.cpp - Chip8::drawing()

```
void Chip8::drawing() {
    if (opcode == 0x00E0) {
        std::memset(gfx, 0, sizeof(gfx));
        drawFlag = true;
        pc = static_cast<uint16_t>(pc + 2);
        return;
    }

    const uint8_t x_register = static_cast<uint8_t>((opcode & 0x0F00) >> 8);
    const uint8_t y_register = static_cast<uint8_t>((opcode & 0x00F0) >> 4);
    const uint8_t height = static_cast<uint8_t>(opcode & 0x000F);
    const uint8_t x_position = V[x_register];
    const uint8_t y_position = V[y_register];
    if (!can_read_memory(I, height)) {
        stop("sprite reads outside memory");
        return;
    }

    V[0xF] = 0;
    for (uint8_t row = 0; row < height; ++row) {
        const uint8_t sprite = memory[I + row];
        const uint8_t screen_y = static_cast<uint8_t>((y_position + row) % 32);
        for (uint8_t column = 0; column < 8; ++column) {
            if ((sprite & (0x80 >> column)) == 0) {
                continue;
            }
            const uint8_t screen_x = static_cast<uint8_t>((x_position + column) % 64);
            const uint16_t pixel = static_cast<uint16_t>(screen_y * 64 + screen_x);
            if (gfx[pixel] == 1) {
                V[0xF] = 1;
            }
            gfx[pixel] ^= 1;
        }
    }

    drawFlag = true;
    pc = static_cast<uint16_t>(pc + 2);
}
```

CHIP-8 draws **sprites**: N rows of 8 pixels, where each row is one byte in memory starting at address **I**. The draw operation uses **XOR**, not overwrite:

- For each row the code reads `memory[I + row]`, and for each of its 8 bits (tested with the mask `0x80 >> column`) it XORs the corresponding screen pixel.
- If a pixel that was already lit is turned off by the XOR, **VF** is set to 1. Games use this **collision flag** to detect that a ball or bullet hit something. Drawing the same sprite twice at the same place erases it, which is how movement is animated.

- The screen index is `screen_y * 64 + screen_x`. The starting coordinates come from `VX` and `VY` and each pixel position is wrapped with `% 64` and `% 32`.
- Before drawing, `can_read_memory(I, height)` guards against a sprite that would read past the end of memory.
- `drawFlag` is set so the frontend knows the screen changed, and `pc` advances by 2.

Verified example. The three-instruction program `V0 = 0; V1 = 0; I = font(V0); DRW V0, V1, 5 (6000 6100 F029 D015)` draws the built-in glyph for the digit 0 at the top-left corner. The framebuffer, printed as text after running the core headlessly:

```
####
#.#
#.#
#.#
####
```

This matches the font bytes `F0 90 90 90 F0` from Section 5.1: `F0 = ####`, `90 = #.#`.

7.6 Keypad Instructions

Listing 12: Chip8.cpp - Chip8::keypad()

```
void Chip8::keypad() {
    const uint8_t x = static_cast<uint8_t>((opcode & 0xF00) >> 8);

    if ((opcode & 0xF000) == 0xE000) {
        const uint8_t key_index = V[x];
        if (key_index >= 16) {
            stop("EX instruction used an invalid key index");
            return;
        }

        if ((opcode & 0x00FF) == 0x009E) {
            pc = static_cast<uint16_t>(pc + (key[key_index] ? 4 : 2));
            return;
        }
        if ((opcode & 0x00FF) == 0x00A1) {
            pc = static_cast<uint16_t>(pc + (key[key_index] ? 2 : 4));
            return;
        }
        stop("invalid keypad instruction");
        return;
    }

    if (opcode == static_cast<uint16_t>(0xF00A | (x << 8))) {
        for (uint8_t key_index = 0; key_index < 16; ++key_index) {
            if (key[key_index]) {
                V[x] = key_index;
                pc = static_cast<uint16_t>(pc + 2);
                return;
            }
        }
        return;
    }

    stop("invalid keypad instruction");
}
```

EX9E and **EXA1** read the key number from `VX` and skip the next instruction depending on whether that key is held in `key[]`. A key value of 16 or more is invalid and stops the CPU. **FX0A** scans keys 0-F; if one is down it stores the index in `VX` and advances. If **no** key is down the function returns **without changing `pc`**. The same **FX0A** therefore executes again on the next cycle, which is how "wait for a key" is realised without any blocking call. Timers keep ticking while the CPU waits, because the frontend still calls `update_timers()`.

7.7 Index Register and Memory Instructions

Listing 13: Chip8Memory.cpp - Chip8::memory_instructions() (excerpt)

```
void Chip8::memory_instructions() {
    const uint8_t x = static_cast<uint8_t>((opcode & 0xF00) >> 8);

    if ((opcode & 0xF000) == 0xA000) {
        I = opcode & 0xFFFF;
        pc = static_cast<uint16_t>(pc + 2);
        return;
    }

    switch (opcode & 0x00FF) {
    case 0x001E:
        I = static_cast<uint16_t>(I + V[x]);
        pc = static_cast<uint16_t>(pc + 2);
        break;
    case 0x0029:
        if (V[x] > 0x0F) {
            stop("FX29 received a non-hex digit");
            return;
        }
        I = static_cast<uint16_t>(0x050 + (V[x] * 5));
        pc = static_cast<uint16_t>(pc + 2);
        break;
    case 0x0033:
        if (!can_read_memory(I, 3)) {
            stop("FX33 writes outside memory");
            return;
        }
        memory[I] = static_cast<uint8_t>(V[x] / 100);
        memory[I + 1] = static_cast<uint8_t>((V[x] / 10) % 10);
        memory[I + 2] = static_cast<uint8_t>(V[x] % 10);
        pc = static_cast<uint16_t>(pc + 2);
        break;
    ... // FX55 and FX65: register store / load loops (bounds-checked)
    }
}
```

The remaining cases follow the same pattern (**FX55**/**FX65** loop from register 0 to X and copy between **V[]** and **memory[I..]**, each after a bounds check).

- **ANNN** sets the index register **I**. It is the usual first step before drawing a sprite.
- **FX1E** adds **VX** to **I** (used to step through tables).
- **FX29** points **I** at the font glyph for the hex digit in **VX**. Each glyph is 5 bytes and the font begins at 0x050, so the address is **0x050 + VX * 5**. Values above 0xF stop the CPU.
- **FX33** converts **VX** to decimal digits: hundreds **V/100**, tens **(V/10)%10**, ones **V%10**, stored in three consecutive bytes at **I**. Games use it to print scores.
- **FX55** / **FX65** save and restore registers **V0..VX** to and from memory, so subroutines can preserve state.

7.8 Timers and Random Numbers

Listing 14: Chip8Timers.cpp - timer opcodes and 60 Hz tick

```
void Chip8::timer_instructions() {
    const uint8_t x = static_cast<uint8_t>((opcode & 0x0F00) >> 8);

    switch (opcode & 0x00FF) {
        case 0x0007:
            V[x] = delay_timer;
            break;
        case 0x0015:
            delay_timer = V[x];
            break;
        case 0x0018:
            sound_timer = V[x];
            break;
        default:
            stop("invalid timer instruction");
            return;
    }
    pc = static_cast<uint16_t>(pc + 2);
}

void Chip8::update_timers() {
    if (delay_timer > 0) {
        --delay_timer;
    }
    if (sound_timer > 0) {
        --sound_timer;
    }
}
```

There are two independent 8-bit timers. The **delay timer** is read with **FX07** and written with **FX15**; games use it for timing events. The **sound timer** is written with **FX18**; the buzzer sounds while it is non-zero. **update_timers()** is not an opcode: it decrements each non-zero timer by one and must be called at 60 Hz. The core deliberately does not measure time itself, leaving that to the frontend (Section 9).

Listing 15: Chip8Execution.cpp - CXNN

```
void Chip8::random_number() {
    const uint8_t x = static_cast<uint8_t>((opcode & 0x0F00) >> 8);
    const uint8_t nn = static_cast<uint8_t>(opcode & 0x00FF);
    V[x] = static_cast<uint8_t>((std::rand() & 0xFF) & nn);
    pc = static_cast<uint16_t>(pc + 2);
}
```

CXNN generates a random byte and masks it with **NN**, so **C0 0F** gives a value 0-15. Games use it for enemy movement, random pieces and similar.

8. Execution Walkthroughs

To confirm that the core behaves as described, the four core files (**Chip8.cpp**, **Chip8Execution.cpp**, **Chip8Memory.cpp**, **Chip8Timers.cpp**) were compiled without modification together with a small headless test driver that calls **load()** and **emulate_cycle()** and prints the machine state after each instruction. The following outputs are real.

8.1 IBM Logo ROM: First 14 Instructions

roms/Ibm.ch8 is the standard first test ROM for a new emulator. It clears the screen, then draws the logo in four sprite pieces.

Table 9: Trace of roms/lbm.ch8

#	PC	Opcode	Instruction	Result
1	200	00E0	CLS	Screen cleared, PC = 202
2	202	A22A	LD I, 0x22A	I = 0x22A (address of logo sprite data)
3	204	600C	LD V0, 0x0C	V0 = 12 (x position)
4	206	6108	LD V1, 0x08	V1 = 8 (y position)
5	208	D01F	DRW V0, V1, 15	Draw 15-row sprite from I at (12, 8)
6	20A	7009	ADD V0, 9	V0 = 0x15
7	20C	A239	LD I, 0x239	I = 0x239 (next sprite piece)
8	20E	D01F	DRW V0, V1, 15	Draw second piece
9	210	A248	LD I, 0x248	I = 0x248
10	212	7008	ADD V0, 8	V0 = 0x1D
11	214	D01F	DRW V0, V1, 15	Draw third piece
12	216	7004	ADD V0, 4	V0 = 0x21
13	218	A257	LD I, 0x257	I = 0x257
14	21A	D01F	DRW V0, V1, 15	Draw fourth piece

Note the pattern the CPU follows: load an address into **I**, position with **v0/v1**, then draw. After 200 cycles the framebuffer (printed with **#** for lit pixels) shows the logo. The horizontal stripes are part of the sprite data:

[illegible]

(Rows 8-22 of the 32-row display; all other rows are blank.)

8.2 Call and Return

A hand-assembled 10-byte program shows how 2NNN and 00EE use the stack. Program: 2206 (CALL 0x206), 7001 (ADD V0,1), 1204 (JP 0x204, an idle loop), 6005 (LD V0,5), 00EE (RET).

Table 10: Call / return trace

#	PC before	Opcode	PC after	SP	V0	What happened
1	200	2206	206	1	00	CALL: pushed 0x200, jumped to 0x206
2	206	6005	208	1	05	V0 = 5 inside the subroutine
3	208	00EE	202	0	05	RET: pc = stack[0] + 2 = 0x202
4	202	7001	204	0	06	Continues after the CALL; V0 = 6
5	204	1204	204	0	06	JP to itself (idle loop)

8.3 Carry Flag

Program 60FF 6102 8014: load V0 = 255, V1 = 2, then add V1 to V0.

Table 11: 8XY4 carry trace

#	Opcode	V0	V1	VF	Comment
1	60FF	FF	00	0	V0 = 255
2	6102	FF	02	0	V1 = 2
3	8014	01	02	1	255 + 2 = 257 = 0x101; low 8 bits stored, carry flag set

9. Timing and Frontend Integration

A real CHIP-8 executes a few hundred instructions per second while its timers tick at exactly 60 Hz. The core has no clock, so the frontend keeps two **accumulators** that convert elapsed real time into a number of CPU cycles and timer ticks.

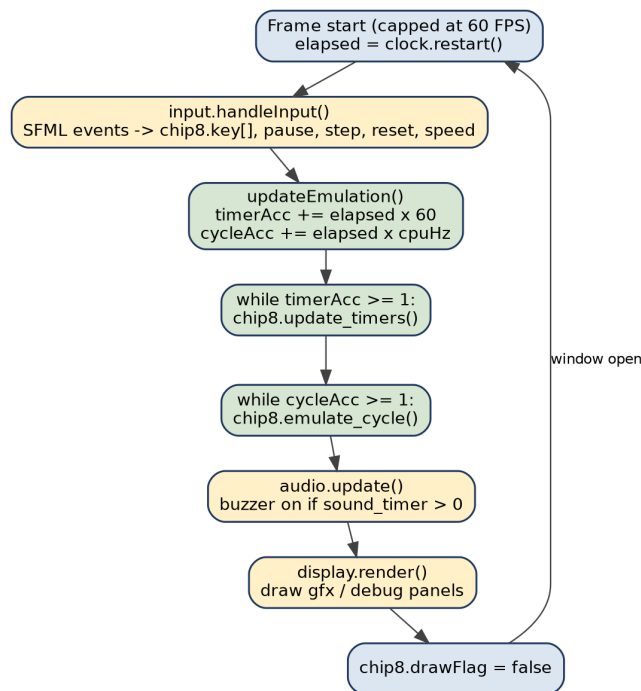


Figure 4: One frame of the main loop (main.cpp)

Listing 16: Frontend.cpp - the running branch of updateEmulation()

```

} else {
    state.stepRequests = 0;
    // Cap catch-up work after a window drag / debugger stall to 100 ms.
    elapsedSeconds = std::clamp(elapsedSeconds, 0.0, 0.1);
    state.cycleAccumulator += elapsedSeconds * state.cpuHz;
    state.timerAccumulator += elapsedSeconds * 60.0;

    // These ticks belong to elapsed time before this frame's instructions.
    // A newly written ST=1 must survive until the next tick to make a beep.
    while (state.timerAccumulator >= 1.0 && chip8.running) {
        chip8.update_timers();
        state.timerAccumulator -= 1.0;
    }
    while (state.cycleAccumulator >= 1.0 && chip8.running) {
        chip8.emulate_cycle();
        state.cycleAccumulator -= 1.0;
    }
}
}

```

- **Two rates.** `cycleAccumulator` grows by `elapsed × cpuHz` (700 by default) and `timerAccumulator` by `elapsed × 60`. Each whole unit triggers one call. Fractions carry over to the next frame, so the average speed is accurate even though frames are 16.7 ms apart.
- **Timers first.** Timer ticks are applied before the frame's instructions so that a freshly written `sound_timer = 1` survives long enough to be heard.
- **Catch-up cap.** `elapsedSeconds` is clamped to 0.1 s, so dragging the window or stopping in a debugger does not cause a burst of thousands of instructions.
- **Pause and step.** When paused nothing runs; the F2 key runs exactly one `emulate_cycle()` while timers stay frozen, giving predictable single-stepping.
- **Errors.** If the core sets `running = false`, the frontend shows a "CORE STOPPED" message; F5 reloads the ROM.

10. SFML Graphical Frontend

The frontend turns the core's state into pixels, sound and events. It contains no opcode logic.

Table 12: Frontend components

Component	Role
Display	Creates a fixed 1366x768 borderless SFML window (60 FPS limit). Play mode shows the game scaled 21x. Debug mode shows the game at 9x beside live registers V0-VF, PC, I, SP, the current opcode, timers, the keypad and per-game controls. Text uses a built-in pixel font, so no font files are needed.
Input	Polls SFML events, maps physical keys to <code>chip8.key[]</code> , and handles Space (pause), F1 (mode), F2 (step), F5 (reset), +/- (speed), Esc (minimise). Releasing focus clears all held keys.
Audio	Generates a 480 Hz square wave in memory (no sound file) and loops it while <code>sound_timer > 0</code> , unless paused or unfocused.
Controls	Loads up to six lines of per-game key hints from <code>controls/<game>.txt</code> for the debug panel.

The drawing of the CHIP-8 screen is a simple loop over the 2048-entry `gfx` array; each lit pixel becomes a scaled square:

Listing 17: `Display.cpp - drawScreen()` (core loop)

```
void Display::drawScreen(sf::RenderTarget& target, const Chip8& chip8,
                        float x, float y, float width, float height)
{
    int scale = static_cast<int>(std::min(width / 64, height / 32));
    if (scale < 1) {
        return;
    }
    float left = std::floor(x + (width - 64 * scale) / 2);
    float top = std::floor(y + (height - 32 * scale) / 2);
    rectangle(target, left, top, 64.0f * scale, 32.0f * scale, screenColor);
    sf::VertexArray pixels(sf::PrimitiveType::Triangles);
    // Read exactly the existing 2048 framebuffer entries. No sprite/opcode work.
    for (int row = 0; row < 32; ++row) {
        for (int column = 0; column < 64; ++column) {
            if (chip8.gfx[row * 64 + column] != 0) {
                addBlock(pixels, left + column * scale, top + row * scale,
                        static_cast<float>(scale), static_cast<float>(scale), accent);
            }
        }
    }
    target.draw(pixels);
}
```

Because `Display` receives a `const Chip8&`, it can only read state. The main loop (Figure 4) simply repeats: handle input, update emulation, update audio, render.

11. Testing and ROM Compatibility

- **Integration tests** (`tests/FrontendTests.cpp`, about 640 lines) use small synthetic programs, not downloaded games. According to the README they cover all 16 key mappings, simultaneous keys, focus loss, mode switching, pause/resume, single-stepping, CPU-rate limits, 60 Hz timers, key waits, reset/reload failures, mouse controls, controls-file parsing and pixel scaling, and they save layout screenshots.
- **Error-path tests** deliberately trigger a core fault and a missing ROM to check that errors are reported cleanly.
- **ROM library.** The `roms/` folder holds 33 ROM files (Pong, Tetris, Breakout, Invaders, Tank, Connect4, Maze, and more). The ROM README records a smoke check: each retained ROM ran for 60,000 instructions in 18 input scenarios without a core stop. One ROM (`X-mirror.ch8`) was removed because it uses the unsupported `ONNN` opcode and halts at PC 0x204.
- The README states plainly that the smoke check is not a full gameplay validation.

In this report's own verification, the core compiled cleanly without SFML and correctly ran the IBM logo ROM and the hand-written call, carry and draw programs.

12. Design Notes, Quirks and Limitations

CHIP-8 interpreters differ in a few well-known "quirks", and ROMs from different eras expect different behaviour. This table records how this emulator behaves.

Table 13: Behaviour choices and quirks

Area	Behaviour in this emulator	Comment
Shifts 8XY6 / 8XYE	Shift VX in place; VY is ignored	CHIP-48 / SUPER-CHIP style. The original COSMAC VIP copied VY first. Many ROMs written for CHIP-48 / SUPER-CHIP expect the in-place version.
FX55 / FX65	Leave I unchanged	Modern behaviour. The original incremented I; a few old ROMs depend on that.
FX1E	No overflow flag	Only a very small number of ROMs depend on an overflow flag here.
DXYN wrapping	Sprite pixels wrap around screen edges	Some interpreters clip at the edge instead.
FX0A	Accepts a key as soon as it is down (does not wait for release)	Works for the tested ROMs; the strict original waits for release.
ONNN	Rejected, CPU stops	Native machine-code calls cannot be emulated; almost no ROM uses them.
SUPER-CHIP / XO-CHIP	Not implemented	Standard 64x32 CHIP-8 only.
Platform	Fixed 1366x768 layout; minimise uses the Windows API	The core is portable; the frontend layout is Windows-oriented.

Strengths

- **Clean separation** between the SFML-free core and the frontend; the core can be tested headlessly.
- **Defensive programming.** Every memory access, stack operation, jump and register index is bounds-checked; failures stop the virtual CPU with a clear message instead of crashing.

- **Correct flag handling** through saved operand copies in the ALU code.
- **Excellent debugging tools:** live register view, single-step, adjustable CPU rate and an opcode display.

Possible Future Improvements

- Add a quirks switch (VY-shift, I-increment for FX55/65, sprite clipping) so a wider range of ROMs run correctly.
- Make the frontend layout resizable and remove Windows-only calls so it builds cleanly on Linux and macOS.
- Add opcode-level unit tests for the core (for example, every 8XY_ case and boundary conditions for FX33/FX55) alongside the frontend tests.
- Implement SUPER-CHIP extensions (128x64 display, scrolling, larger font).
- Add save-states and an instruction disassembler in the debug view.

13. Conclusion

The project delivers a complete CHIP-8 emulator: a compact, well-organised core that implements 34 opcodes with careful bounds checking, plus an SFML frontend that offers both a play view and a debugger. The core follows the textbook fetch-decode-execute loop, decodes with simple bit masks, and keeps all machine state in one class, so the behaviour of any instruction can be traced from opcode to effect in a few lines of code. Traces of real programs confirm that call/return, arithmetic with carry, and XOR sprite drawing behave as the specification requires. The frontend keeps CPU speed and the 60 Hz timers independent of frame rate and never modifies the emulation logic.

The main limitations, such as the fixed Windows-oriented layout and a single fixed set of quirks, are engineering choices rather than flaws in the core design, and the architecture makes them straightforward to improve.

References

- Project repository: <https://github.com/awaiskamil-dev/chip8-emulator>
- T. Greene (Cowgod), "Cowgod's Chip-8 Technical Reference v1.0" - widely used opcode reference.
- CHIP-8 (Wikipedia article) - history and instruction-set overview.
- SFML 3 documentation: <https://www.sfml-dev.org>
- ROM source referenced in the repository:
<https://github.com/netpro2k/Chip8/tree/master/games>